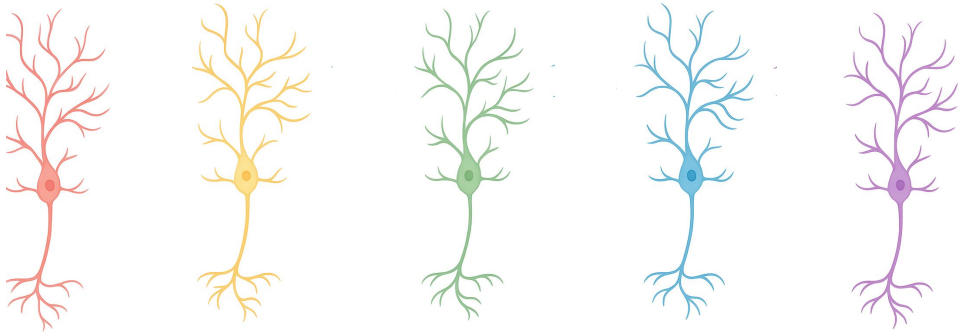
Prairie Voles

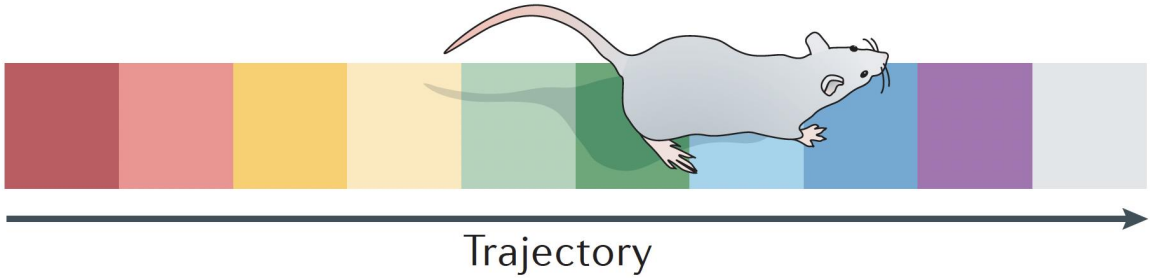
(and the evil code we wrote for them…)

Kelly, Autumn, Audrey

In 2014, John O'Keefe, May-Britt Moser, and Edvard Moser shared the Nobel Prize for their discoveries of place cells and grid cells

Hippocampal place cells

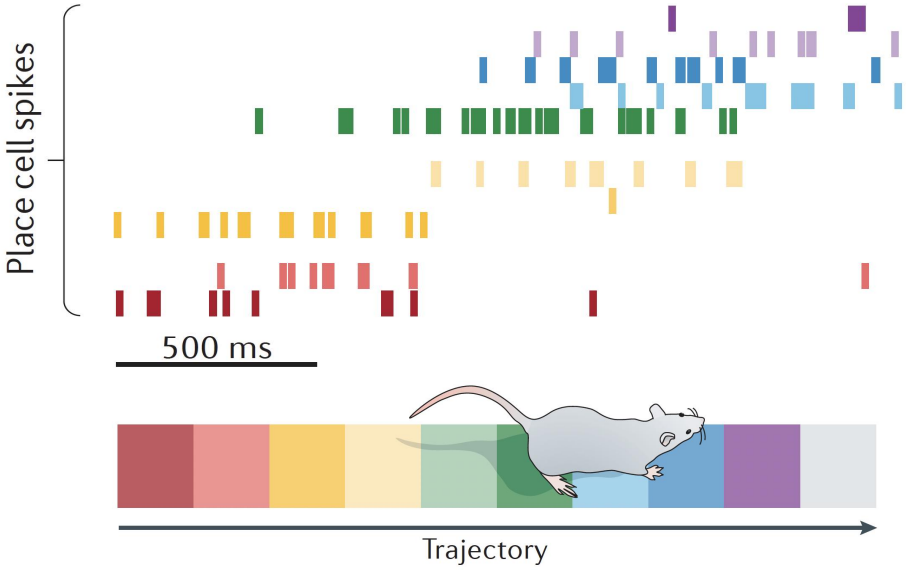




Cheese

Laura Colgin, *Nature Review Neuroscience*, 2016

In 2014, John O'Keefe, May-Britt Moser, and Edvard Moser shared the Nobel Prize for their discoveries of place cells and grid cells



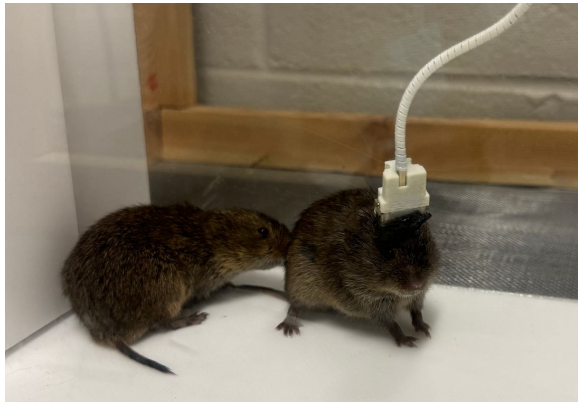
Active exploration

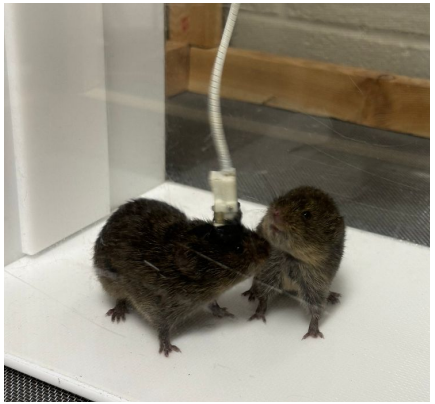
Making Memories

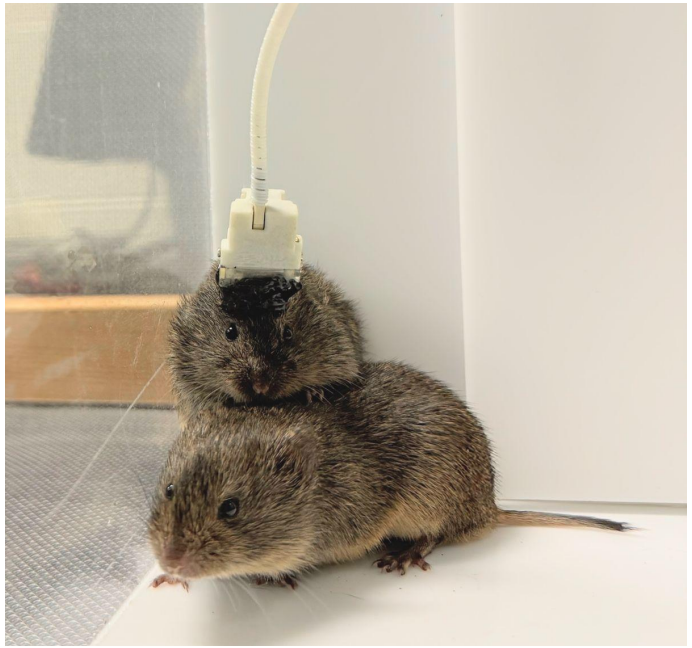
Sharp Wave Ripple

At rest

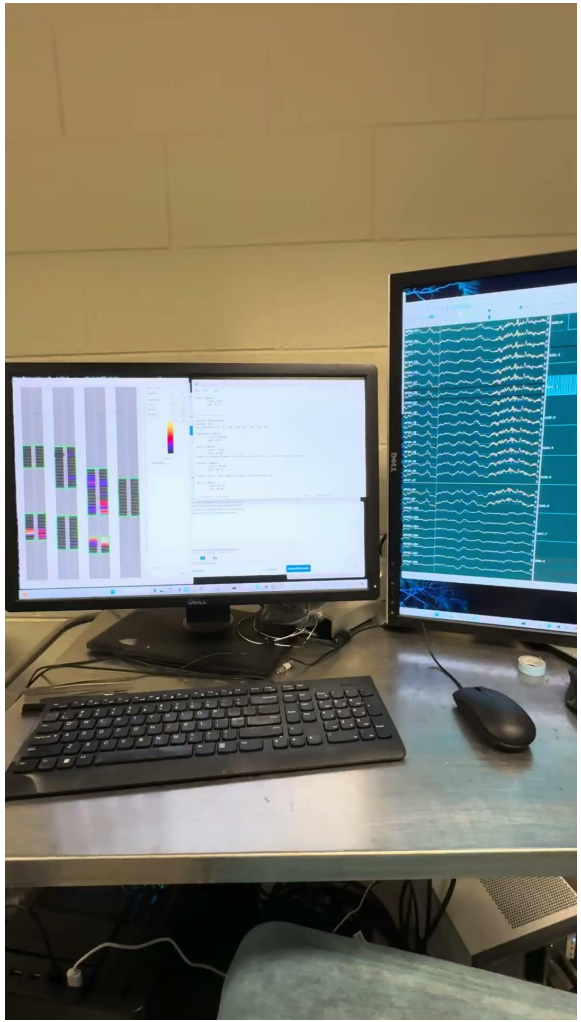
Is spiking activity sequential for social interactions?







get sniffed sniff cuddle



SWR.csv

start	peak	stop
100	150	300
450	520	550
…	…	…

behavior.csv

behavior	start	stop
social interaction	1	50
social interaction	60	70
sleeping	90	305
sleeping	445	560
…	…	…

spike_times.npy

1
1.01
1.02
1.03
…

cluster_ID.npy

2
4
23
5
…

cluster_KSLabel.tsv

1	good
2	good
3	mua
…	…

Step 1: Familiarize ourselves with the data products (provided by Kelly) and reading the data into a Python environment

Step 2: Re-organizing our data into accessible data structure(s) for easier access during analysis (i.e., dataframe, object, etc.)

Step 3: Write a function that will automatically search through data for co-activity during social interaction / sharp wave ripples

Step 4: Write visualization routines to help analyze neuron / cell assembly behavior

Step 5: Attempt to algorithmically predict / classify interactions based solely on neuron data

This was our initial project proposal…

…but how much did we *really* accomplish?

Step 1: Familiarize ourselves with the data products (provided by Kelly) and reading the data into a Python environment

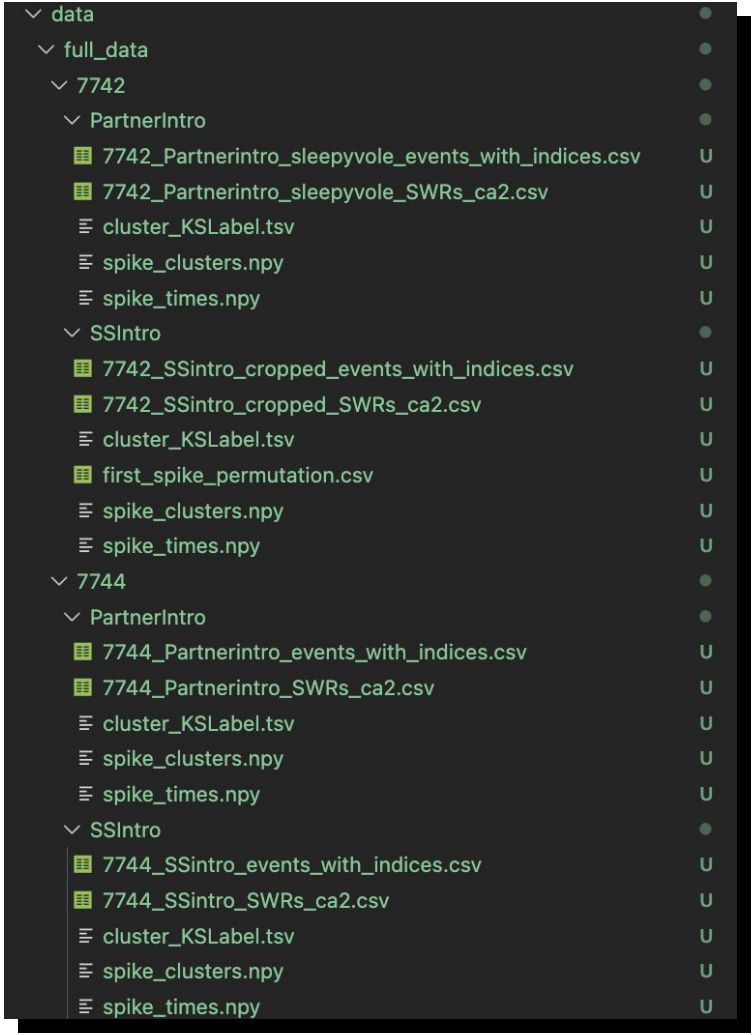
Step 2: Re-organizing our data into accessible data structure(s) for easier access during analysis (i.e., dataframe, object, etc.)

loading_utils.py

Is capable of…

- Loading data into Pandas DataFrames
- Merging datasets based on times
- Filtering DataFrames for specific values

  (example: good clusters only)



Step 3: Write a function that will automatically search through data for co-activity during social interaction / sharp wave ripples

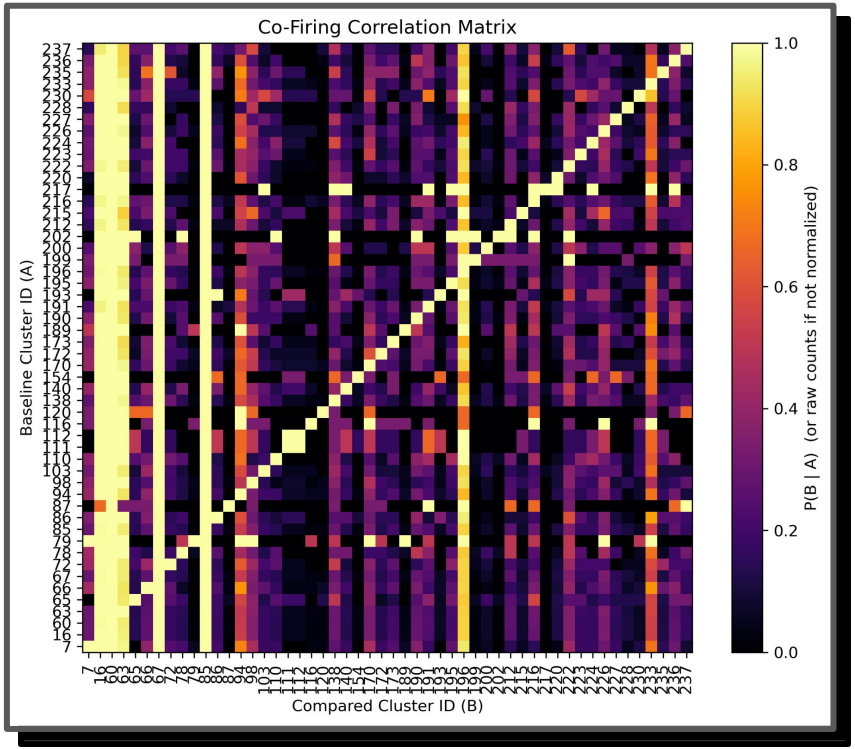
analysis_utils.py

sequence_matching_utils.py

Are capable of…

- Making matrices which represent the probability of Neuron B | Neuron A

“Unnormalized” Sequence Matches



Step 3: Write a function that will automatically search through data for co-activity during social interaction / sharp wave ripples

analysis_utils.py

sequence_matching_utils.py

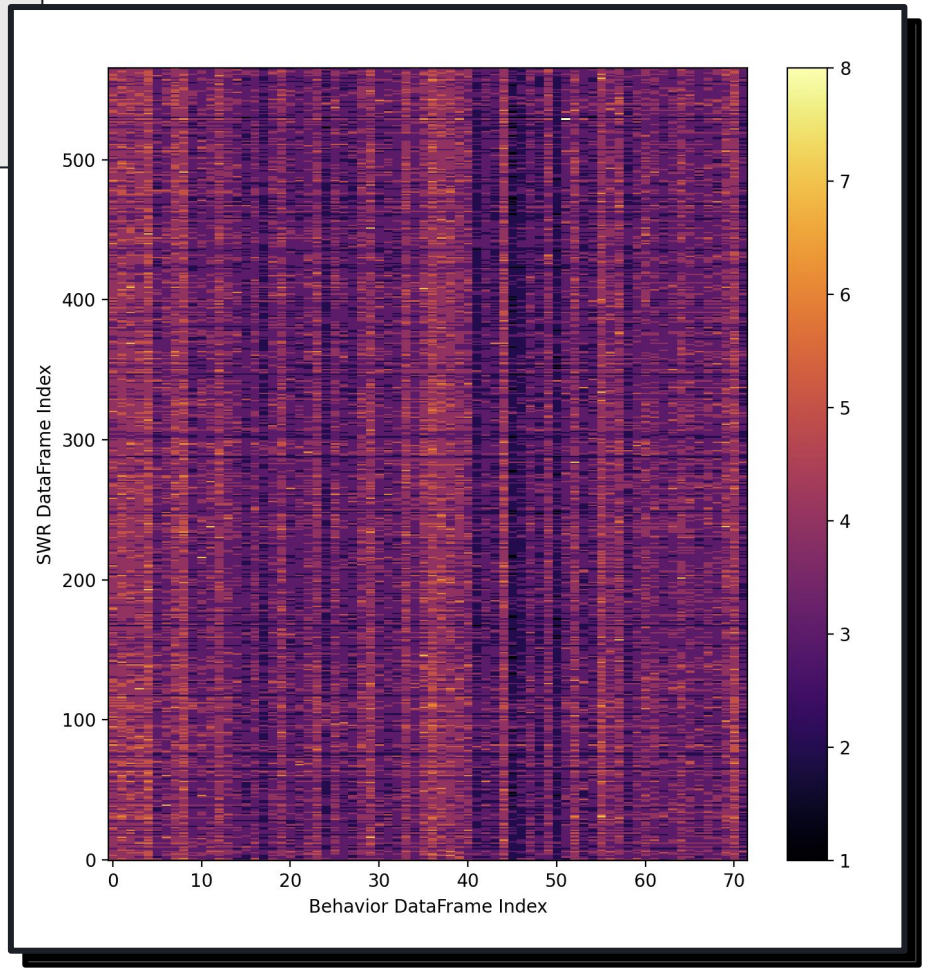
Are capable of…

- Making matrices which represent the probability of Neuron B | Neuron A
- Identifying sequences of neuron clusters that fire in two separate DataFrames

Future Direction…

- Save time of long matches and check video for clues of what might be replaying

“Unnormalized” Sequence Matches

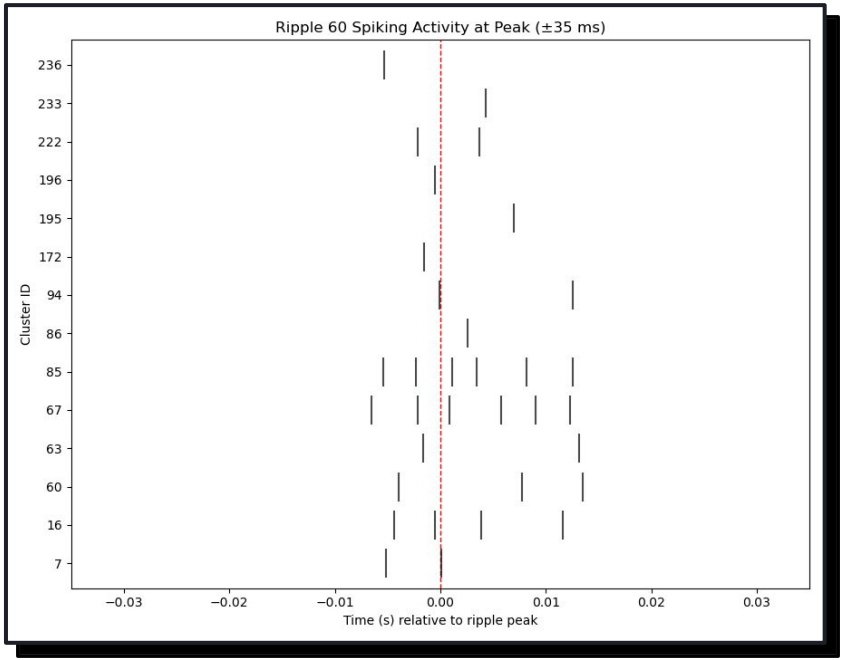


Step 4: Write visualization routines to help analyze neuron / cell assembly behavior

raster_plot_utils.py

Are capable of…

- Making raster plots for activity during individual SWRs or ‘Mega raster’ of all SWRs



Step 4: Write visualization routines to help analyze neuron / cell assembly behavior

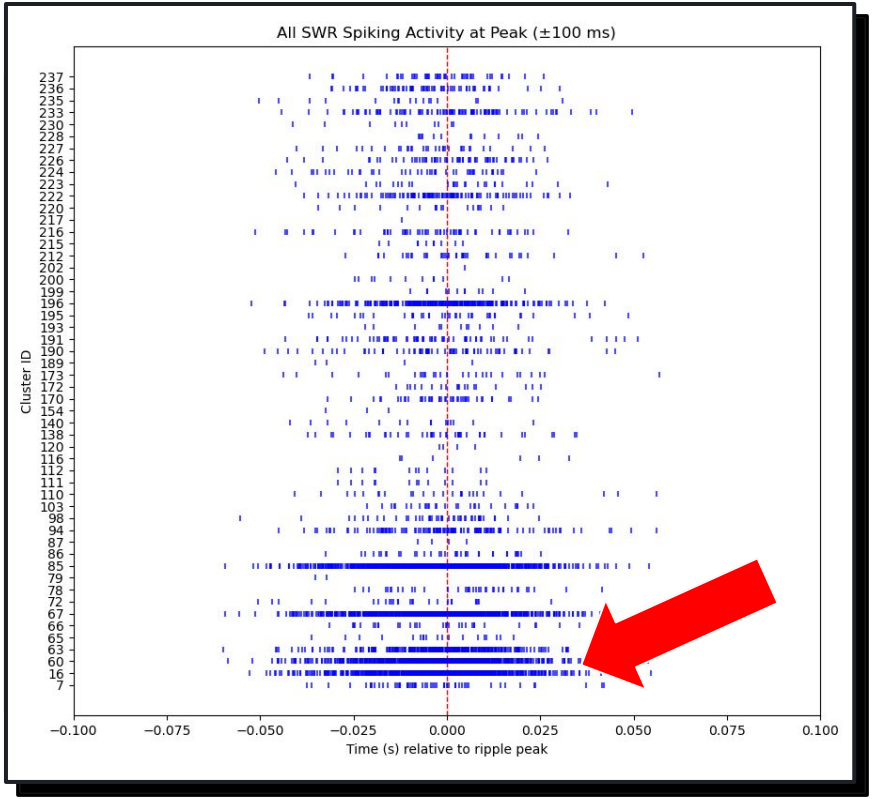
raster_plot_utils.py

Are capable of…

- Making raster plots for activity during individual SWRs or ‘Mega raster’ of all SWRs

WHAT IS THAT??

IS THAT NORMAL??



Step 1: Familiarize ourselves with the data products (provided by Kelly) and reading the data into a Python environment

Step 2: Re-organizing our data into accessible data structure(s) for easier access during analysis (i.e., dataframe, object, etc.)

Step 3: Write a function that will automatically search through data for co-activity during social interaction / sharp wave ripples

Step 4: Write visualization routines to help analyze neuron / cell assembly behavior

Step 5: Attempt to algorithmically predict / classify interactions based solely on neuron data

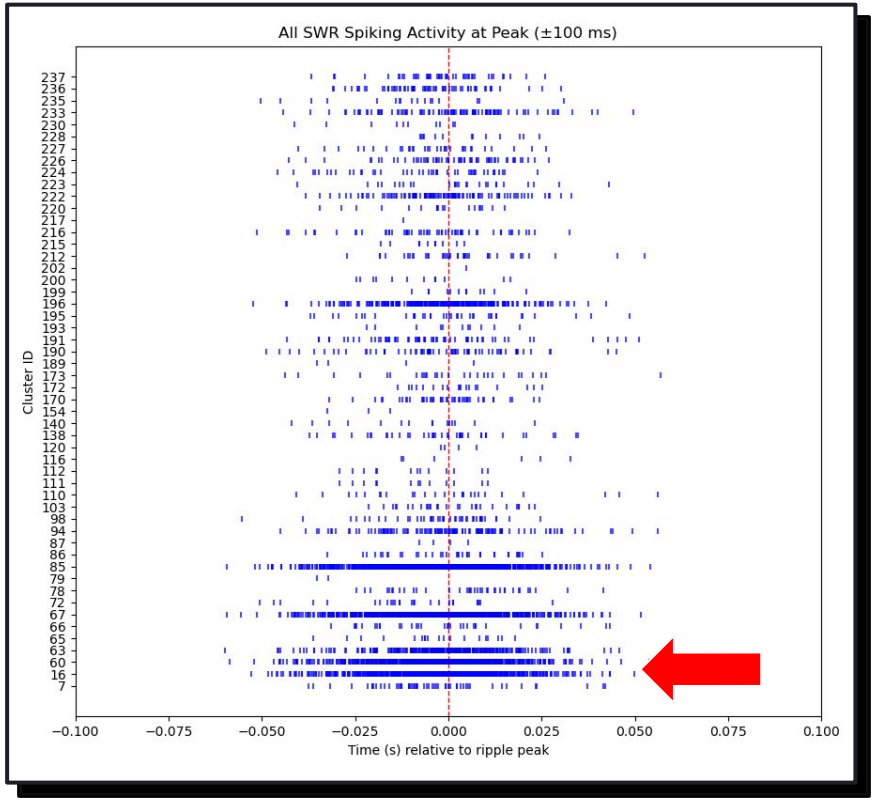
Step 4: Write visualization routines to help analyze neuron / cell assembly behavior

raster_plot_utils.py

analysis_utils.py

Are capable of…

- Making raster plots for activity during individual SWRs or ‘Mega raster’ of all SWRs



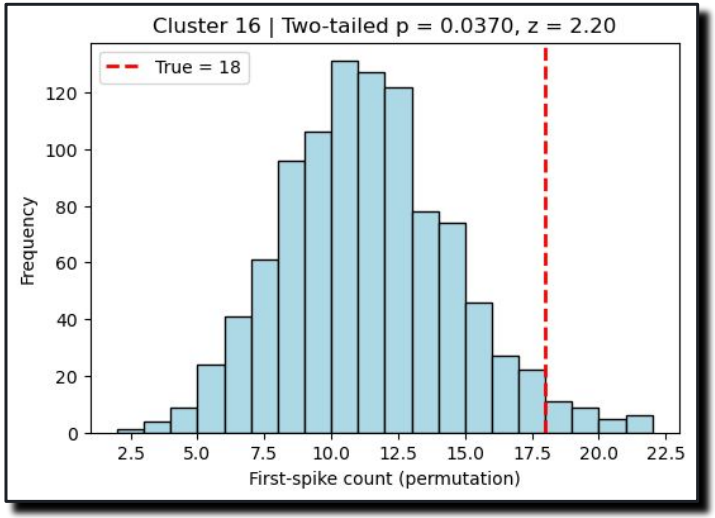
Step 4: Write visualization routines to help analyze neuron / cell assembly behavior

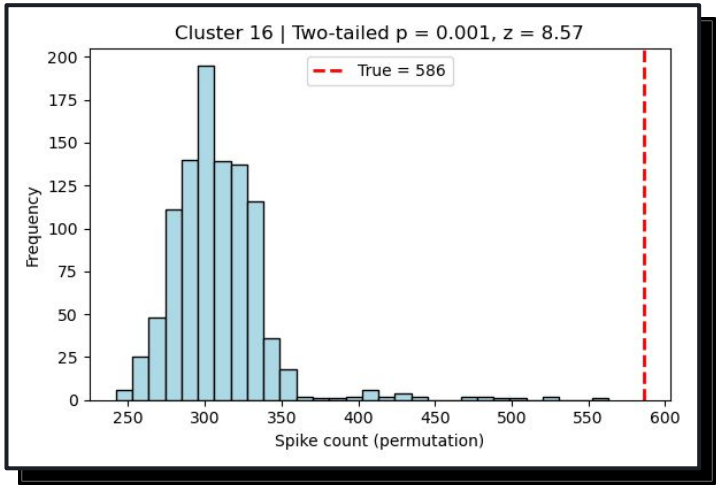
raster_plot_utils.py

analysis_utils.py

Are capable of…

- Making raster plots for activity during individual SWRs or ‘Mega raster’ of all SWRs
- Identify whether a neuron is spiking more than expected within a SWR
- In 1000 random shifts the SWR window to see if this is different than expected





Want to run it yourself?

Then *these files* are here to support you!

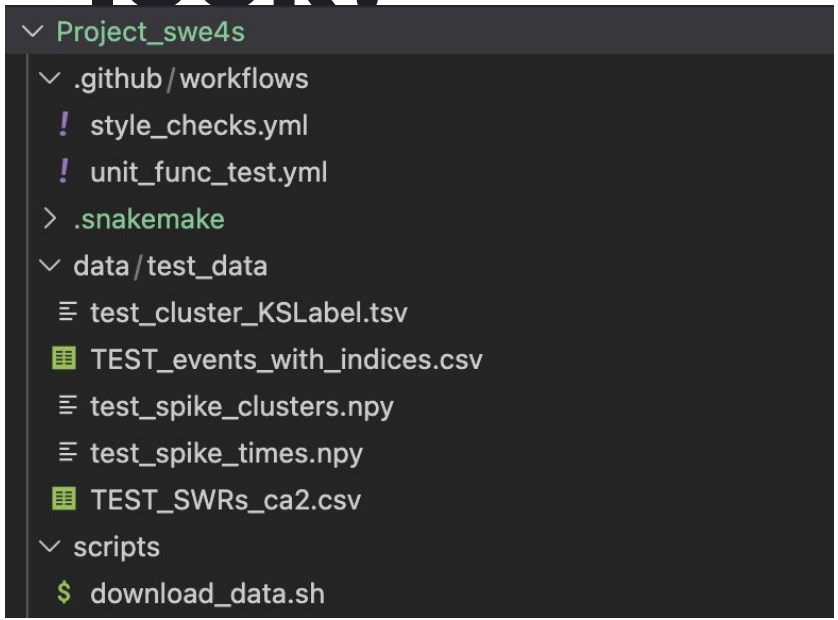
Future Directions?

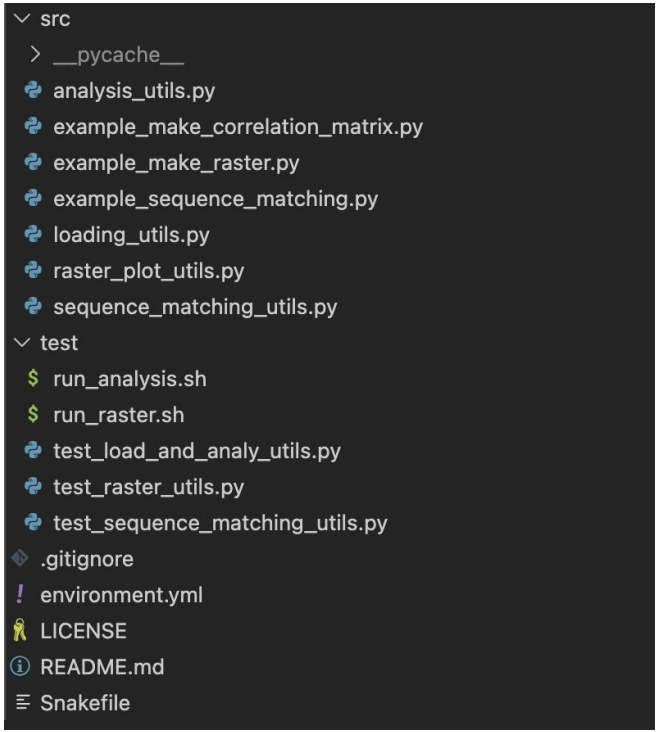
KELLY

It sure would be convenient to see if these functions could run any faster…

Also, I wonder if additional filtering for neuron types could provide unique insight into the sequence behavior!

How does the code look?





How are we testing it?

Unit Testing - Loading and Analysis Utils

test_load_and_analy_utils.py

- analysis_utils.py and loading_utils.py
- Responsible for testing:
  - Edge cases such as empty lists, missing columns and invalid modes
  - Circular permutation functional flow tests
  - Datatypes and output structures
  - Missing data errors
  - Bad file path errors

```
def test_match_times_empty_window(self):
    # Make SWR file with bad times
    fake_swr = pd.DataFrame({"Start": [99999], "Stop": [100000]})
    out = lu.group_dataframes_by_time(
        window_df=fake_swr,
        event_df=self.spike_df,
        event_time_column="Time",
        keep_event_columns=["Time"],
        time_interval_columns=["Start", "Stop"],
        progress=False
    )
    self.assertEqual(out.iloc[0]["Event Times"], [])
```

Unit Testing - Sequence Matching Utils

Test_sequence_match_utils.py

- Sequence_match_utils.py
- Responsible for Testing:
  - Correct handling of absent values
  - Correct computation
  - Hashing of lists and arrays
  - Normalization accuracy
  - Correct compute overlap matrix

```python
def test_prefix_hash(self):

    valid_array = [0, 4, 20, 200, 3]

    # Ensures that '_prefix_hash' works with lists and np.arrays
    self.assertTrue(
        utils._prefix_hash(valid_array, BASE, MOD),
        utils._prefix_hash(np.array(valid_array), BASE, MOD),
    )

    # Ensures that an empty list does return a hash (0)
    self.assertEqual(utils._prefix_hash([], BASE, MOD), [0])
```

Unit Testing - Raster Plots

test_raster_utils.py

- raster_plot_utils.py
- Responsible for testing:
  - Correct long form exploding
  - Correct t_rel calculation
  - Handling of string list inputs
  - Ripple filtering
  - Incorrect datatype handling
  - Invalid ripple index handling

```python
def test_select_all_ripples(self):
    # ripple_index=None should return the full exploded df
    out = rpu.select_ripples_to_plot(self.exp_df, ripple_index=None)
    self.assertEqual(len(out), len(self.exp_df))

def test_select_single_ripple(self):
    # Selecting a single ripple index returns only those rows
    out = rpu.select_ripples_to_plot(self.exp_df, ripple_index=0)
    self.assertEqual(set(out["ripple_idx"]), {0})
    # In test data ripple 0 has 3 spikes
    self.assertEqual(len(out), 3)

def test_select_multiple_ripples(self):
    # Multiple ripple indices returns a union of their rows
    out = rpu.select_ripples_to_plot(self.exp_df, ripple_index=[0, 1])
    self.assertEqual(set(out["ripple_idx"]), {0, 1})
    self.assertEqual(len(out), len(self.exp_df))
```

Functional Testing

```
valid_call ran in 5 sec with 2/16 lines to STDERR/OUT
Comparing output file to expected output file...
 PASS EXIT CODE (LINE 15)
```

run_analysis.sh

- Functional test for loading and analysis pipeline
- Uses example_make_correlation_matrix.py
  - Loads spike data
  - Match time (spike during a SWR)
  - Outputs correlation matrix
  - Visualize correlation matrix
- Checks output against expected data

run_raster.py

- Functional test for raster plot raster_plot_utils.py
- Uses example_make_raster.py
  - Load spike data
  - Match time (spike during a SWR)
  - Construct exploded raster dataframe
  - Apply ripple selection
  - Visualize raster plot
- Checks output against expected data